

◆ DEVELOPER BUILD · SHIP

# Kubernetes Troubleshooting Agent

## what we're going to build

A LangGraph agent that investigates broken pods on a **real cluster**, diagnoses the root cause with gpt-4o, and proposes a fix behind a **human-approval gate**. This deck is the engineering plan — architecture, the graph, the tools, and how we run it.

LangGraph

LangChain tools

gpt-4o

kubernetes client

Streamlit

kind

# What **we**, the developers, are building

READ (auto)

## Live cluster access

A read-only layer over the Kubernetes API — list unhealthy pods, describe them, tail logs. Live-first: it uses a real cluster whenever one is reachable.

REASON

## A LangGraph agent

A state machine that investigates in a ReAct loop, then emits a **structured** diagnosis (failure class · root cause · evidence · fix · action).

GATE

## Human-in-the-loop

The graph **pauses** at `interrupt()` for approval before any change. Dry-run by default.

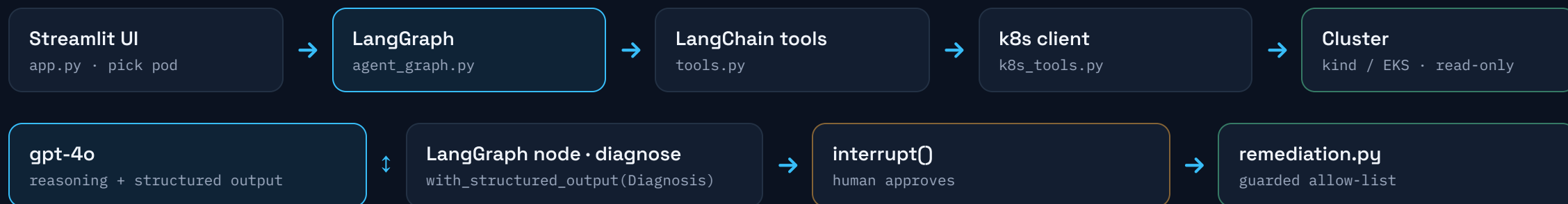
ACT (guarded)

## Remediation allow-list

One tiny, safe set of actions: `rollout restart` · `scale` · `delete pod`. No arbitrary kubectl.

**Design rule:** the LLM only reads and reasons. Exactly one module can mutate the cluster, and only after a human clicks approve. Everything else is read-only.

# How the pieces fit together



**Data flow:** UI → graph → tools read the cluster → gpt-4o diagnoses → graph pauses for approval → the guarded module applies exactly one action. State lives in a MemorySaver checkpoint.

# The agent is a state machine

```
# agent_graph.py - nodes & edges
g = StateGraph(State)
g.add_node("investigate", investigate) # LLM + tools
g.add_node("tools", ToolNode(TOOLS))
g.add_node("diagnose", diagnose) # structured
g.add_node("approval_gate", approval_gate)
g.add_node("remediate", remediate)

g.add_conditional_edges("investigate",
    tools_condition, {"tools": "tools", END: "diagnose"})
g.add_edge("tools", "investigate") # ReAct loop
g.add_conditional_edges("diagnose", after_diagnose)
g.compile(checkpointer=MemorySaver())
```

- ▶ **investigate**  $\Rightarrow$  **tools** — the agent decides which read tools to call, and loops until it has evidence.
- ▶ **diagnose** — one structured LLM call  $\rightarrow$  a typed **Diagnosis** (Pydantic).
- ▶ **after\_diagnose** — if the fix is **manual**, stop; else go to the gate.
- ▶ **approval\_gate** — **interrupt()** pauses the graph; the UI resumes with **Command(resume=...)**.
- ▶ **remediate** — runs the one approved action, dry-run unless **execute** is on.

# The files we write — and **real** failures to test on

| FILE                        | RESPONSIBILITY                      |
|-----------------------------|-------------------------------------|
| <code>agent_graph.py</code> | the LangGraph orchestration         |
| <code>tools.py</code>       | read-only LangChain tools           |
| <code>k8s_tools.py</code>   | live cluster access + mock fallback |
| <code>remediation.py</code> | guarded write allow-list            |
| <code>app.py</code>         | Streamlit UI drives the graph       |
| <code>rbac.yaml</code>      | least-privilege access              |

## Live cases we deploy to break on purpose

Real workloads in `live-cases/live-cases.yaml` — the agent reads them live, nothing is canned.

- ▶ **CrashLoopBackOff** — app exits, `DATABASE_URL` unset
- ▶ **ImagePullBackOff** — image tag doesn't exist
- ▶ **OOMKilled** — 250M alloc vs a 150Mi limit
- ▶ **Pending** — requests 64 CPUs, unschedulable
- ▶ **CreateContainerConfigError** — missing Secret

# The build plan, step by step

## STEP 1

### Cluster + cases

`setup-cluster.sh` → a real `kind` cluster with the live failures applied.

## STEP 2

### Read layer

Wrap the k8s client in read-only tools; auto-detect live vs mock.

## STEP 3

### The graph

Investigate ⇌ tools → diagnose → gate → remediate, with a checkpoint.

## STEP 4

### UI + gate

Streamlit drives the graph; approval resumes it via `Command`.

```
# run the whole thing
pip install -r requirements.txt && cp env.sample .env # add OPENAI_API_KEY
./live-cases/setup-cluster.sh # real cluster + broken pods
streamlit run app.py # agent auto-detects the live cluster
```

**Definition of done:** pick a real broken pod → the agent investigates live, names the root cause with evidence, proposes a fix → a human approves → the guarded action runs on the cluster.